

Claude Code — Complete Prompt Handbook

Project: E-Commerce REST API | Spring Boot + MySQL

Table of Contents

CLAUDE CODE IN ACTION — First Prompts	
SECTION 3 — CLAUDE.md & Project Setup	
Lecture 3.1 — Context Engineering: The Core Idea.....	
Lecture 3.2 — Provide Context through CLAUDE.md and Rules.....	
Lecture 3.3 — Auto Memory: Claude's Personal Notebook	
Lecture 3.4 — Git from Day 1	
SECTION 4 — Generating the E-Commerce API	
Lecture 4.1 — Before You Generate	
Lecture 4.2 — Prompt 1: Data Layer	
Lecture 4.3 — Prompt 2: Business Layer	
Lecture 4.4 — Unit Tests	
Lecture 4.5 — API Testing with Swagger UI	
SECTION 5 — Code Review & Refinement	
Lecture 5.1 — Review the Data Layer	
Lecture 5.2 — Review the Business Layer	
Lecture 5.3 — Refinement Prompts	
Lecture 5.4 — Feature Workflow: Plan & Build Product Reviews	
SECTION 6 — Context Management & Commands	
Lecture 6.1 — Context Usage with /context and /compact.....	
Lecture 6.2 — Session Management: Resume & Continue	
Lecture 6.3 — Fresh Context with /clear	
Lecture 6.4 — Rewind: Ctrl+Z for Your Conversation	
SECTION 7 — Skills & Custom Commands.....	
Lecture 7.1 — Skills	
Lecture 7.2 — Skill Creator Plugin.....	
SECTION 8 — Hooks: Zero-Token Automation.....	
Lecture 8.1 — Register Hooks.....	
Lecture 8.2 — Test the Hooks	
SECTION 9 — Subagents	
Lecture 9.1 — Integration Tests	
Lecture 9.2 — Create the Test Writer Subagent	
Lecture 9.3 — Delegate Controller Tests	

Lecture 9.4 — Parallel Controller Test Agents	
SECTION 10 — MCP, Connectors & Plugins	
Lecture 10.1 — Playwright Connector (MCP).....	
Lecture 10.2 — MCP Scope, Limitations & CodeMode	
Lecture 10.3 — Plugins: Install Context7	
SECTION 11 — Git, Docker & CI/CD	
Lecture 11.1 — GitHub: Push	
Lecture 11.2 — Resolve a GitHub Issue End-to-End	
Lecture 11.3 — Dockerfile	
Lecture 11.4 — Docker Compose	
Lecture 11.5 — GitHub Actions CI/CD	
SECTION 12 — Bonus: Build a Next.js Storefront	
Lecture 12.1 — Create the Storefront Project	
Lecture 12.2 — Frontend CLAUDE.md	
Lecture 12.3 — Plan the Storefront	
Lecture 12.4 — Enable CORS on the API	
Lecture 12.5 — Quality Gates	
Lecture 12.6 — Full-Stack Deploy	
Lecture 12.7 — Revisit Auto Memory	

CLAUDE CODE IN ACTION — First Prompts

A quick warm-up before the project: four tiny Java programs, each in a different mode. Switch modes with Shift+Tab. Use a throwaway folder.

PROMPT — Self-Check

List the tools you have access to, tell me your current model, and read this directory and tell me what you see.

PROMPT — Hello World (Manual mode)

Create HelloWorld.java that prints a welcome message, then compile and run it.

PROMPT — FizzBuzz (Accept Edits mode)

Create FizzBuzz.java that prints 1 to 30 with Fizz, Buzz, and FizzBuzz, then run it.

PROMPT — Primes (Plan mode)

Plan a PrimeNumbers.java that prints all primes up to 100 using the Sieve of Eratosthenes. Ultrathink about the most efficient approach.

PROMPT — Fibonacci (Auto mode)

Create Fibonacci.java that prints the first 15 Fibonacci numbers, then run it.

SECTION 3 — CLAUDE.md & Project Setup

Lecture 3.1 — Context Engineering: The Core Idea

Everything in this course is one idea applied repeatedly: control what Claude has in its context window.

Tip: More context is not better. A bloated context window costs tokens on every prompt and dilutes Claude's focus. The skill is giving Claude exactly what the current task needs — no more.

Lecture 3.2 — Provide Context through CLAUDE.md and Rules

CLAUDE.md is read automatically every session, so you never repeat your conventions. Commit it so every teammate inherits the same rules.

PROMPT 3.2a — Verify CLAUDE.md Is Working

1. What annotation do we use for dependency injection?
2. What type do we use for money fields?

Tip: For existing projects you did not create from scratch — run `/init` instead of writing `CLAUDE.md` manually. Claude reads the entire codebase and generates a `CLAUDE.md` from what is actually there. Use `/init` later, after a project has grown beyond its original `CLAUDE.md`. For this course, Prompts 2.1 and 2.2 above are the right approach — we know our conventions upfront.

PROMPT 3.2b — Verify the Rules Are Loaded

What should the API error response look like according to standard conventions?

Lecture 3.3 — Auto Memory: Claude's Personal Notebook

Claude Code maintains its own memory file as you work, built from what it observes across sessions — separate from the `CLAUDE.md` you write by hand.

PROMPT 3.3 — Inspect Auto Memory

What do you have in memory for this project?

Tip: Auto Memory is machine-local — stored under `~/.claude/projects/[project-path]/` on your machine, never in the repo, never committed to git. Each developer builds their own independent memory. `CLAUDE.md` is the shared layer (committed); Auto Memory is the personal layer (local only). Claude Code writes to it sparingly, so early on it is often empty — that is normal. It accumulates slowly and rarely prunes outdated facts, so review it occasionally.

Lecture 3.4 — Git from Day 1

Note: Set up Git now — before any Java code is written. Not at the end of the course.

PROMPT 3.4a — Review and Complete .gitignore

Review the existing `.gitignore` for `ecommerce-api`.
Make sure it covers everything that should be ignored for this project:
Spring Boot, Maven, MySQL, VS Code, and Claude Code.
Add anything missing. Show exactly what you added and why.

PROMPT 3.4b — Initialize Git

Set up git for `ecommerce-api`.
Initialise the repo, stage all files, and make the first commit with a conventional commit message.

Tip: Commit after every major generation. `git commit` is your rollback point. If the generation goes wrong, `git reset` brings you back. Without commits you have no safety net.

SECTION 4 — Generating the E-Commerce API

We split generation into two prompts — the data layer first, then the business layer on top — with a checkpoint between to review, compile, and correct.

Lecture 4.1 — Before You Generate

Checklist before running either prompt:

Initializr project is open in IDE

CLAUDE.md exists in project root

db/schema.sql exists in the project

schema.sql has been run in MySQL Workbench (ecommerce_db exists with all tables)

Git is initialised with at least one commit

Lecture 4.2 — Prompt 1: Data Layer

This prompt generates the structural layer: enums, entities, repositories, exception classes, and DTOs — no business logic yet.

PROMPT 4.2 — Generate the Data Layer

Generate the complete data layer for ecommerce-api. Read db/schema.sql first.

Application config:

- MySQL at localhost:3306/ecommerce_db, username root, password root
- ddl-auto=validate (tables already exist), SQL logging on

Three enums needed:

OrderStatus: PENDING, CONFIRMED, PROCESSING, SHIPPED, DELIVERED, CANCELLED, REFUNDED

PaymentMethod: CREDIT_CARD, DEBIT_CARD, UPI, NET_BANKING, WALLET, CASH_ON_DELIVERY

PaymentStatus: PENDING, COMPLETED, FAILED, REFUNDED

Two entity business rules:

- Order number is generated on save as 'ORD-' followed by 8 uppercase UUID characters
- OrderItem subtotal is calculated automatically as unit price × quantity
- Saving an order must cascade to its items and payment

TERMINAL — Commit

```
git add -A
git commit -m 'feat: add data layer (enums, entities, repositories, DTOs, exceptions)'
```

Lecture 4.3 — Prompt 2: Business Layer

With the structure in place, this prompt generates the services and controllers that wire it all together.

PROMPT 4.3 — Generate the Business Layer

Generate the service and controller layers for ecommerce-api.

Three business rules to apply:

- Stock reduces when an order is placed, not at shipment
- Cancelling an order restores stock for all items;
if payment was completed, mark it REFUNDED
- Order status transitions are one-way:
 - PENDING → CONFIRMED or CANCELLED
 - CONFIRMED → PROCESSING or CANCELLED
 - PROCESSING → SHIPPED or CANCELLED
 - SHIPPED → DELIVERED
 - DELIVERED, CANCELLED, and REFUNDED are final

TERMINAL — Commit

```
git add -A
git commit -m 'feat: add service and controller layers'
```

Lecture 4.4 — Unit Tests

The app starts — now prove the service logic is correct with fast unit tests, before code review.

PROMPT 4.4 — Generate Unit Tests for All Services

Generate unit tests for all four services.
Additional business scenarios to cover:

- Category create: duplicate name
- Product create: duplicate SKU
- Customer register: duplicate email
- Order placeOrder: customer not found, insufficient stock
- Order cancelOrder: already delivered, already cancelled

Tip: If a test fails, paste the exact assertion error back to Claude Code. It knows what it generated and can fix the mismatch immediately. A failing test is useful information — either the expectation is wrong, or the service has a bug.

Lecture 4.5 — API Testing with Swagger UI

One dependency gives you an interactive API browser at localhost:8080/swagger-ui.html — no Postman needed.

PROMPT 4.5 — Add Springdoc OpenAPI

Add interactive Swagger UI to ecommerce-api using the latest springdoc-openapi dependency.

Tip: Run through all 12 endpoints in Swagger UI before the Section 5 code review. Test the happy path for each one. Any 500 errors at this stage point to service or controller bugs — fix them now before the review section.

SECTION 5 — Code Review & Refinement

Lecture 5.1 — Review the Data Layer

PROMPT 5.1 — Review Order Entity

Review Order.java.

Explain to me:

1. How `orderNumber` is generated — walk through the `@PrePersist` method
2. Why `@Getter` `@Setter` instead of `@Data` on an entity
3. How `CascadeType.ALL` on `OrderItems` works when we call `orderRepository.save()`
4. Why `FetchType.LAZY` matters here and what problem `EAGER` would cause
5. Why `@Enumerated(EnumType.STRING)` is safer than `ORDINAL`

Lecture 5.2 — Review the Business Layer

PROMPT 5.2 — Review OrderService.placeOrder()

Walk through `OrderService.placeOrder()` line by line.

1. Customer lookup — what happens if the customer does not exist?
2. Stock reduction loop — which service method is called and what does it throw?
3. Total amount calculation — when is `OrderItem.subtotal` set?
4. Payment creation — why `PENDING` status at the point of order creation?

Lecture 5.3 — Refinement Prompts

REFINEMENT 2 — Low Stock Check on Order

In `OrderService.placeOrder()`, after all stock reductions are done, check if any product's remaining stock is below 10.

If so, log a warning: 'Low stock alert: [productName] has [remaining] units left'.

Do not throw an exception — this is a warning only.

Lecture 5.4 — Feature Workflow: Plan & Build Product Reviews

Adding a feature is the natural place to practise planning mode. Here you add Product Reviews end to end: plan it, review the plan, then approve it and let Claude build the feature.

Planning mode lets Claude propose a step-by-step plan without editing a single file, so you review and correct it before anything changes. Toggle it with `Shift+Tab`, or just ask.

PROMPT 5.4 — Plan the Product Reviews Feature (plan mode)

Enter plan mode.

Add a Product Reviews feature: a customer can post a review — a rating from 1 to 5 and a comment — on a product, and list all reviews for a product.

Approve the plan and Claude implements the whole feature — the `ProductReview` entity, repository, DTOs, service, controller endpoints, and exception handling — all following `CLAUDE.md`. Reuse this plan-and-approve pattern for every new feature.

Tip: For a large or shared feature, you can ask Claude to write the approved plan to a spec file first (`docs/specs/...`) so teammates review it before any code. For a feature this size, approving the plan is enough.

SECTION 6 — Context Management & Commands

Context is large now — the right moment to learn context management.

Lecture 6.1 — Context Usage with `/context` and `/compact`

PROMPT 6.1a — Check Context After Generation

```
# Check context window usage (tokens consumed vs available):  
/context
```

Tip: Run `/context` right after the big generation prompts — it shows how much of the window is consumed and the headroom remaining. That headroom is what tells you when to `/compact`.

PROMPT 6.1b — Compact a Long Session

```
/compact
```

```
After compact, verify Claude still knows the project:  
What do you know about the project?
```

When to use each:

`/context` after any large generation — see context window usage

`/compact` after 3-4 refinement prompts — compress without losing context

Always verify after `/compact` — ask Claude one fact about the project

Lecture 6.2 — Session Management: Resume & Continue

Every session has an ID and saved history: resume any recent session, or continue the most recent one automatically.

TERMINAL — Resume a previous session

```
# List recent sessions with timestamps:  
claude --resume  
  
# Resume a specific session by ID:  
claude --resume <session-id>  
  
# Resume most recent session automatically:  
claude --continue
```

Tip: Resume when you were mid-task and had to stop — same context, same history. Continue (`claude --continue`) reopens the most recent session. Start fresh (`/clear`) when the task has completely changed.

Lecture 6.3 — Fresh Context with `/clear`

When you switch focus entirely — for example, moving from building the API to writing tests — start with a clean slate so the old conversation does not leak into the new task.

PROMPT 6.3 — Fresh Context Before Test Writing

```
/clear
```

Use `/clear` when the focus changes completely; reach for `/compact` instead when you want to keep the thread but free up space.

Lecture 6.4 — Rewind: Ctrl+Z for Your Conversation

Rewind is undo for your whole session — both the code and the conversation. Claude Code checkpoints automatically before every change, so there is nothing to set up. You can implement something, decide you do not want it, and roll the whole thing back.

PROMPT 6.4 — Implement a Feature to Roll Back

```
Add a simple endpoint: GET /api/products/count that returns the total
number of products as a plain number.
```

TERMINAL — Roll it back with rewind

```
# In the Claude Code session, run:
/rewind

# (or press Esc twice)
# Select the checkpoint from before the change.
# The code and the conversation both return to that state.
```

Tip: Rewind undoes the conversation and the file changes together. Git commit is still your durable rollback — rewind is the fast, in-session undo.

SECTION 7 — Skills & Custom Commands

Lecture 7.1 — Skills

Skills are `.md` files with YAML frontmatter in `.claude/skills/` — they can restrict which tools Claude uses and auto-invoke from their description.

Lecture 7.2 — Skill Creator Plugin

TERMINAL — Install Skill Creator

```
# In Claude Code session:
/plugins
# Discover -> search 'skill-creator' -> Install -> restart session
```

PROMPT 7.2a — Use Skill Creator to Build a Skill

```
/skill-creator
I want to create a production readiness check skill for this project.
```

PROMPT 7.2b — Test the Skill

```
/production-readiness-check
```

SECTION 8 — Hooks: Zero-Token Automation

Hooks run automatically on every tool call at zero token cost. Four types: PreToolUse (exit code 2 blocks the tool), PostToolUse, Notification, and Stop.

This is where the two .claude settings files come in — both start empty and fill as you register hooks.

Tip: .claude/settings.json is shared project config — commit this to the repo. It holds tool permissions, MCP server configs, and team-wide hook registrations. Every developer inherits these when they pull the repo. .claude/settings.local.json is your personal machine config — gitignored by default, never committed. Personal API keys, personal hook overrides, anything machine-specific goes here. The split is intentional: shared config in settings.json, personal setup in settings.local.json. A third file, ~/.claude.json, is your user-global config — it holds user-scope MCP servers (claude mcp add --scope user), installed plugins, claude config settings, and auth. The CLI manages it; you never hand-edit it, and it lives outside any repo.

Lecture 8.1 — Register Hooks

PROMPT 8.1 — Register the Hooks

Register the hooks in the right settings file:

In .claude/settings.json (shared, committed — every teammate gets it):
PreToolUse on Edit and Write tools -> .claude/hooks/guardrail.sh

In .claude/settings.local.json (personal, gitignored):
PostToolUse on all tools -> .claude/hooks/file-logger.sh
Stop -> .claude/hooks/stop-notify.sh

Tip: Team guardrails go in .claude/settings.json (committed), so anyone who clones the repo gets the hook and its registration automatically — the scripts in .claude/hooks/ are committed too. Personal hooks (a local logger, a desktop notifier) stay in .claude/settings.local.json, which is gitignored.

Lecture 8.2 — Test the Hooks

PROMPT 8.2 — Trigger the Guardrail

Create TestDummy.java in the service package with:
@Autowired private ProductRepository productRepository;

Tip: Expected result: BLOCKED: @Autowired found. Claude physically cannot write the file.

PROMPT — Explain the File-Logger Hook

Explain the file-logger hook.

SECTION 9 — Subagents

Lecture 9.1 — Integration Tests

Integration tests cover the HTTP layer: routing, validation, exception mapping, and response bodies. `@WebMvcTest` loads Spring MVC without a real server or database, and you build a subagent to write them across every controller.

Lecture 9.2 — Create the Test Writer Subagent

Claude Code's `/agents` command creates subagents interactively: you pick the tools and model, and the agent loads your `CLAUDE.md` so it knows the project conventions. Claude also spins up read-only Explore subagents on its own to understand the codebase; here we build one you control.

IN CLAUDE CODE — Create the webmvc-test-writer with `/agents`

```
/agents
```

```
Description: Writes @WebMvcTest integration tests for ecommerce-api
```

Lecture 9.3 — Delegate Controller Tests

PROMPT 9.3 — OrderController Tests via Subagent

```
Use webmvc-test-writer to write @WebMvcTest tests for OrderController.
```

Lecture 9.4 — Parallel Controller Test Agents

PROMPT 9.4a — Three Controller Test Agents in Parallel

```
Run webmvc-test-writer on these three controllers in parallel – one instance each:  
CategoryController  
ProductController  
CustomerController
```

With every controller's tests now written, run the full suite and measure coverage.

PROMPT 9.4b — JaCoCo Coverage Report

```
Add the JaCoCo Maven plugin to pom.xml for test coverage reporting.  
Run ./mvnw clean verify – this compiles, runs every test (unit and the  
new integration tests), and generates the HTML coverage report.
```

```
Read the report and tell me:
```

1. Overall line coverage and branch coverage percentages.
2. Coverage percentage for each service class.
3. Every class below 80% – which specific methods or branches are not covered.
4. For each gap, one sentence describing the test that would cover it.

Tip: JaCoCo ships with Spring Boot's parent pom — you only need to add the plugin block, not a separate dependency. After running `./mvnw clean verify`, the HTML report is at `target/site/jacoco/index.html`. Green = covered, red = not covered, yellow = partially covered branch.

SECTION 10 — MCP, Connectors & Plugins

Lecture 10.1 — Playwright Connector (MCP)

TERMINAL — Add the Playwright connector (once, user scope)

```
# Register the Playwright MCP server once — user scope makes it available
# in every project on this machine:
claude mcp add --scope user playwright npx @playwright/mcp@latest

# Verify it is connected:
claude mcp list          # or run /mcp inside a Claude Code session
```

Tip: `--scope user` writes to `~/.claude.json` — your personal, machine-global config that is not committed to any repo, so the connector follows you across every project. To share a connector with your team instead, use `--scope project`: that writes a `.mcp.json` at the project root, which you commit so everyone gets it on pull.

PROMPT 10.1 — Playwright Smoke Test

```
Use the Playwright connector to open http://localhost:8080/api/v1/products
Confirm the response is valid JSON.
Tell me if the structure matches the ProductResponse DTO fields.
```

Tip: Playwright screenshot may time out on macOS. Fallback: `curl -s http://localhost:8080/api/v1/products | python3 -m json.tool`

Lecture 10.2 — MCP Scope, Limitations & CodeMode

MCP servers attach at three scopes: local (this machine), project (committed, shared with the team), and user (your personal global tools). Keep project scope minimal — every connected server adds tools to the context window.

Tip: Commit only the MCP servers the whole team needs at project scope. Keep experimental or personal servers at local or user scope so they never bloat a teammate's context window.

Lecture 10.3 — Plugins: Install Context7

A plugin bundles MCP servers, skills, subagents, and hooks into one install from a marketplace. Here we install Context7, which gives Claude up-to-date library and framework docs on demand.

IN CLAUDE CODE — Install Context7

```
/plugins
# Discover -> search 'context7' -> Install -> restart the session
```

PROMPT 10.3 — Use Context7

```
Are we using the latest versions of @WebMvcTest and @MockitoBean?
Also, could you confirm the correct import package for each?
```

Tip: Context7 needs no login — it pulls live docs. That matters here: Spring Boot 4 is new, so Context7 keeps Claude working from today's API instead of older training.

SECTION 11 — Git, Docker & CI/CD

Tip: Git was set up in Section 3.4 and you have committed in conventional format throughout the course. Section 11 covers pushing to GitHub, resolving issues, Docker, and the CI pipeline.

Lecture 11.1 — GitHub: Push

Your project is already committed from earlier in the course — now put it on GitHub. With the gh CLI authenticated, Claude creates the repository, adds it as origin, and pushes — all from one prompt, no manual repo creation or git remote commands. Run gh auth login once first if you have not already.

PROMPT 11.1 — Create the Repo and Push

Create a private GitHub repository named ecommerce-api for this project, then push main to it.

Tip: Pushing uses your existing git credentials. Pull requests come next: when Claude resolves a GitHub issue (next lecture) it opens one for you — run gh auth login once so the gh CLI can.

Lecture 11.2 — Resolve a GitHub Issue End-to-End

With the GitHub connector in place, Claude can take an issue and turn it into a pull request: read the issue, locate the relevant code, propose a fix, and open the PR. On a solo project there is no issue waiting, so file one yourself first.

TERMINAL — Create an issue to resolve

```
# Solo project? File the issue yourself so there is one to fix:
gh issue create --title 'Validate review rating is between 1 and 5' --body 'POST
review accepts ratings outside 1-5; reject invalid ratings with a 400.'
# gh prints the new issue number — use it in the prompt below.
```

PROMPT 11.2 — Fix an Issue and Open a PR

Read the open issue in this repository.

Summarise the problem in one line, then plan the fix before changing anything. Implement the fix on a branch named after the issue (fix/issue-<n>), run the tests, and open a pull request that references the issue. Keep the commit message in conventional format.

Tip: Always have Claude branch and run the tests before opening the PR. The issue number in the PR body links them automatically so the issue closes on merge.

TERMINAL — Review and merge the PR

```
# Claude opened the PR; merging stays a human step.
# Review it on GitHub and merge in the browser, or from the terminal:
gh pr merge --squash --delete-branch
```

Tip: Merging is deliberately manual — you review the diff first. `--squash` keeps main's history clean; `--delete-branch` removes the merged fix branch and switches you back to an updated main, so you are ready for the Docker work. Merging closes the linked issue automatically. (If you merge in the browser instead, run `git checkout main && git pull` afterward.)

Lecture 11.3 — Dockerfile

Tip: `spring-boot-starter-actuator` must be in `pom.xml` before building the Docker image. The health check hits `/actuator/health`.

PROMPT 11.3 — Production Dockerfile

Create a production Dockerfile for `ecommerce-api`. Use a multi-stage build — Temurin 21 JDK to build, JRE for the final image — run as a non-root user, and add a container health check against the actuator health endpoint.

Lecture 11.4 — Docker Compose

PROMPT 11.4 — `docker-compose.yml`

Create `docker-compose.yml` and a `.env` for the local stack — the `ecommerce-api` built from its Dockerfile, and a MySQL 8 service. MySQL should load `db/schema.sql` on first start and take its database name and root password from `.env`. Start the API only after MySQL is healthy, and keep secrets in `.env`, not in the compose file. Expose MySQL on host port 3307 so it does not clash with the local MySQL.

TERMINAL — Run the full stack

```
# Build and start the API + MySQL:
docker compose up --build

# In another terminal, check status and try the API:
docker compose ps
# open http://localhost:8080/swagger-ui.html

# Stop and remove the containers:
docker compose down
```

Note: This compose runs the backend (API + MySQL) from the `ecommerce-api` folder. In the Bonus you create a full-stack `docker-compose.yml` in the parent folder — outside both projects — that adds the Next.js storefront alongside these.

Lecture 11.5 — GitHub Actions CI/CD

PROMPT 11.5 — CI Pipeline

Create a GitHub Actions workflow at `.github/workflows/ci.yml` that runs on `push` and `pull_request` to `main`. Add a test job on Java 21 that starts a MySQL service, loads `db/schema.sql`, and runs the Maven tests; and a build job that builds the Docker image and pushes it to the GitHub Container Registry on pushes to `main`.

SECTION 12 — Bonus: Build a Next.js Storefront

A bonus to finish on: apply the whole workflow to a new stack — a Next.js storefront that consumes your Spring Boot API. The point is that the workflow is stack-agnostic.

Work in a parent folder holding both projects side by side; the storefront opens in its own VS Code window and Claude Code session, with the full-stack docker-compose.yml at the parent root.

Lecture 12.1 — Create the Storefront Project

TERMINAL — Create the storefront project (sibling of the backend)

```
# From the parent folder that holds ecommerce-api:
cd ..          # if you are inside ecommerce-api
npx create-next-app@latest storefront --typescript --app --tailwind
```

Lecture 12.2 — Frontend CLAUDE.md

BONUS 2 — Frontend CLAUDE.md

Create a CLAUDE.md in the storefront project root.

Stack: Latest Next.js App Router, TypeScript, Tailwind, fetch for API calls.

Conventions:

- Typed API client in lib/api — one module per resource
- Server components for data fetching; thin components, no business logic
- Money formatted with Intl.NumberFormat
- API base URL from NEXT_PUBLIC_API_URL

Lecture 12.3 — Plan the Storefront

BONUS 1 — Plan the Storefront (plan mode)

Enter plan mode.

I am building a storefront in Next.js (App Router, TypeScript) that consumes our Spring Boot e-commerce API at http://localhost:8080.

Read the OpenAPI spec at /v3/api-docs and propose a plan: pages (product list, product detail, cart, checkout, order confirmation), the components each page needs, and which API endpoints each one calls.

Lecture 12.4 — Enable CORS on the API

BONUS 3 — Enable CORS on the API (Spring side)

Add a single @Configuration class that enables CORS for the storefront origin http://localhost:3000, methods GET, POST, PUT, DELETE, on all /api/** endpoints.

Lecture 12.5 — Quality Gates

BONUS 6 — Quality Gates

With the Playwright connector, go beyond the happy path – test an empty cart, an out-of-stock add, an invalid checkout, and a missing-product page.

Lecture 12.6 — Full-Stack Deploy

BONUS 7 — Full-Stack Deploy

Create docker-compose.yml with three services:

- db: mysql:8.0, db/schema.sql mounted as an init script
- api: build ./ecommerce-api, depends_on db, port 8080:8080
- storefront: build ./storefront, depends_on api, port 3000:3000, NEXT_PUBLIC_API_URL=http://localhost:8080

Add a Dockerfile to the storefront. Then run

docker compose up --build from the parent folder and confirm the storefront talks to the API in containers.

Lecture 12.7 — Revisit Auto Memory

Back in Lecture 2.4 this came back empty. After generating the API, reviewing it, resolving an issue, shipping Docker and CI, and building the storefront, there is real history now — check what Claude has actually recorded.

PROMPT 12.7 — Revisit Auto Memory

Read your current memory:

1. What have you recorded about this project so far?
2. Are there any facts that are incorrect or outdated?
3. What important project knowledge is missing that you should add?